

VOLUME 1

DevOps Engineering Fieldbook

Linux Systems & Operations

Streams, Pipes & Redirection

A beginner-first lesson in where command input and output travel.

Chapter 02 - Streams, Pipes & Redirection

2.1 Một câu hỏi đơn giản: output của command đi đâu?	—
2.2 Hiểu input và output trước khi học thuật ngữ	—
2.3 Standard Input (stdin)	—
2.4 Standard Output - stdout	—
2.5 Standard Error - stderr	—
2.6 File descriptors 0, 1, and 2	—
2.7 Output redirection: >	—
2.8 Append redirection: >>	—
2.9 Input redirection: <	—
2.10 stderr redirection: 2>	—
2.11 Gộp stdout và stderr	—
2.12 Pipes:	—
2.13 Vì sao pipe không phải temporary file?	—
2.14 Exit status khác error output thế nào?	—
2.15 Những lỗi thường gặp và cách debugging	—
2.16 STOP & RECALL	—
2.17 Bài lab có hướng dẫn	—
2.18 Điểm kiểm tra cuối chapter	—

Streams, Pipes & Redirection

Follow text as it enters a command, leaves it and becomes evidence.

CORE

HANDS-ON

PRODUCTION

WORKED EXAMPLE

Luồng thực thi đầy đủ: tách stdout và stderr

Problem. Muốn giữ normal output và diagnostic output thành hai evidence file để đọc riêng.

Input / initial state. Thư mục lab có `colors.txt`; `does-not-exist.txt` chưa tồn tại.

Command.

```
ls colors.txt does-not-exist.txt >
normal.txt 2> errors.txt
```

Execution flow. Shell mở hai destination trước khi command chạy → `ls` ghi tên file có thật qua stdout → `ls` ghi diagnostic cho file thiếu qua stderr.

Intermediate state. `normal.txt` đã nhận một dòng; `errors.txt` đã nhận diagnostic, dù command chỉ chạy một lần.

Output / result.

```
$ cat normal.txt
colors.txt
$ cat errors.txt
ls: cannot access 'does-not-exist.txt': No such file or directory
```

Why. `>` đổi destination của stdout; `2>` đổi destination của descriptor 2, tức stderr. Hai stream vẫn riêng.

Common failure. Dùng `> combined.txt` rồi ngạc nhiên khi diagnostic vẫn hiện trên terminal.

Debugging evidence. Chạy `cat normal.txt`, `cat errors.txt`, rồi kiểm tra exit status ngay sau command. Đừng suy ra routing từ việc text cùng xuất hiện trên một terminal.

Tags: [BEGINNER] [CORE] [HANDS-ON] [PRODUCTION]

2.1 Một câu hỏi đơn giản: output của command đi đâu?

Mở terminal và chạy một command rất nhỏ:

COMMAND

```
printf 'hello from the terminal\n'
```

Bạn thấy một dòng trên màn hình. Nhưng dòng đó đã đi đâu trước khi bạn nhìn thấy nó? Command có luôn phải hiện kết quả trên terminal không, hay ta có thể gửi cùng kết quả vào file?

Hãy làm trong một lab directory có thể bỏ đi:

COMMAND

```
mkdir -p "$HOME/fieldbook-labs/ch02-streams"
cd "$HOME/fieldbook-labs/ch02-streams"
printf 'first line\n' > message.txt
cat message.txt
```

`printf` tạo text. Dấu `>` bảo shell gửi kết quả vào `message.txt`; `cat` đọc file đó rồi hiện text lên terminal. Đây là problem nhỏ của chapter: **text đi vào đâu và đi ra đâu?** Chưa cần nhớ thuật ngữ Linux. Trước hết, ta chỉ phân biệt input source và output destination.

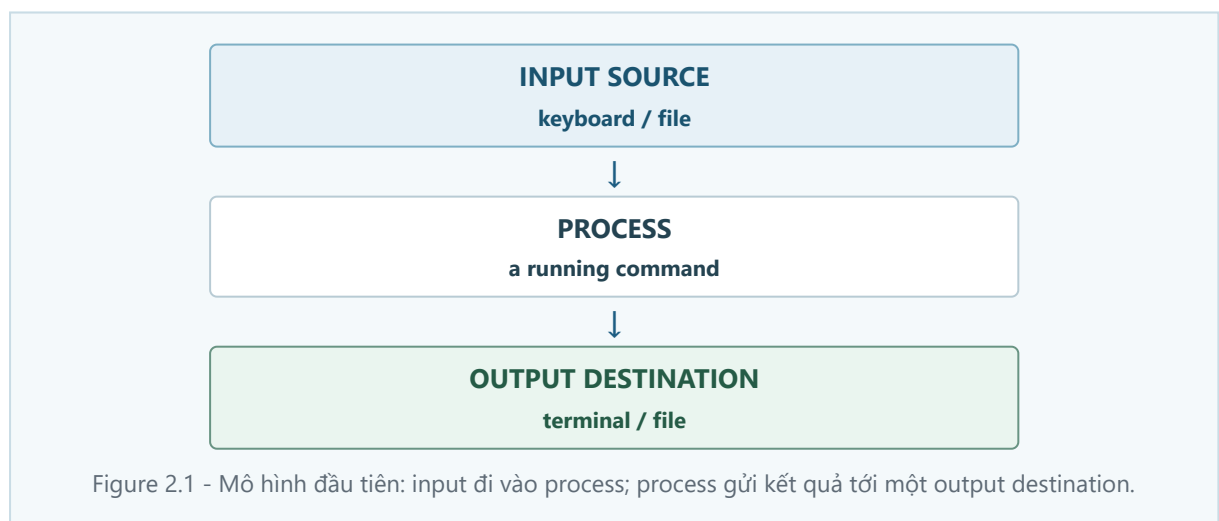
2.2 Hiểu input và output trước khi học thuật ngữ

Một command có thể nhận text từ một nơi và gửi text tới một nơi khác. Hãy tạo file màu, rồi đọc nó:

COMMAND

```
printf 'red\nblue\ngreen\n' > colors.txt
cat colors.txt
wc -l colors.txt
```

`wc -l` đếm số dòng. Ở ví dụ này, bạn đưa filename cho `wc`; command đọc file và in kết quả. Quan sát quan trọng: text không tự "thuộc về màn hình". Terminal chỉ là một destination thường gặp.



Mũi tên luôn đi từ input source tới process, rồi từ process tới output destination. Các phần sau chỉ đặt tên chính xác cho những đường đi này.

2.3 Standard Input (stdin)

Một command có thể là **shell builtin** hoặc external program. `printf` thường là Bash builtin và có thể chạy ngay trong shell process; external program thì thường chạy trong process riêng. Dù là builtin hay external program, các khái niệm stdin, stdout, stderr và redirection vẫn áp dụng như nhau. Các diagram ghi PROCESS minh họa trường hợp external command; ta không cần đi sâu hơn vào implementation của shell ở đây.

Một external program chạy như một **process**: một chương trình đang chạy. Process thường có một đường nhận input mặc định, tên là **standard input (stdin)**.

Thử để `cat` đọc trực tiếp từ terminal:

COMMAND

```
cat
```

Gõ một dòng ngắn rồi nhấn Enter; `cat` lặp lại dòng nó nhận được. Khi `cat` đang đọc stdin từ một interactive terminal, nhấn **Ctrl-D** trên một dòng trống sẽ báo **end-of-input / EOF** cho lần đọc đó. Điều này không có nghĩa keyboard hoặc terminal ngừng nhận input vĩnh viễn; sau khi `cat` kết thúc, shell lại nhận input bình thường.

Lần hai, đổi input source thành file:

COMMAND

```
cat < colors.txt
```

`<` là **input redirection**: shell mở `colors.txt` và nối nó vào stdin của `cat` trước khi `cat` chạy. `cat colors.txt` và `cat < colors.txt` có thể cho cùng text, nhưng cơ chế khác nhau: một bên là filename argument, bên kia là stdin.

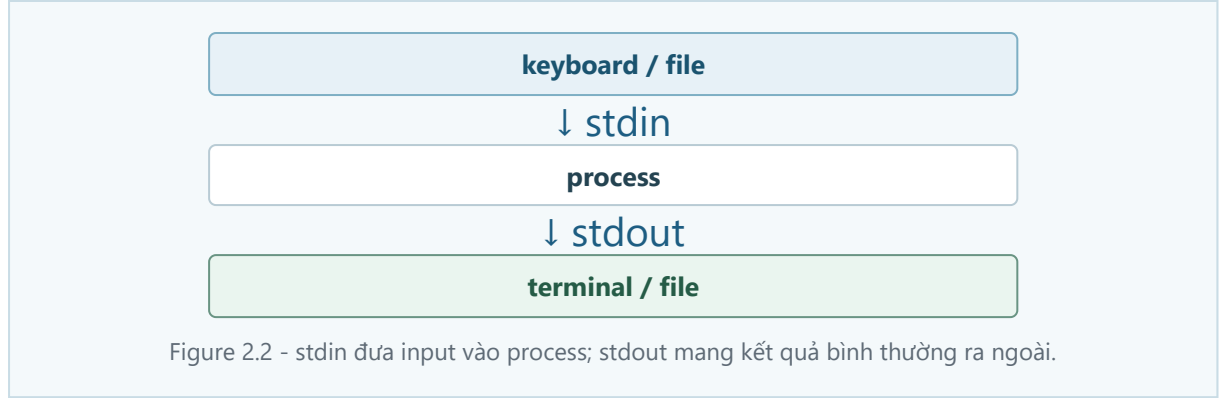
2.4 Standard Output - stdout

Khi một process tạo ra kết quả bình thường, kết quả đó thường đi qua **standard output (stdout)**. Nếu không có redirection, stdout thường đi tới terminal.

COMMAND

```
printf 'one\ntwo\nthree\n'
printf 'one\ntwo\nthree\n' > numbers.txt
wc -l < numbers.txt
```

Experiment thứ hai cho thấy stdout có thể đổi destination mà program không cần biết file tên gì. Shell làm việc nối stdout tới `numbers.txt`; `printf` chỉ ghi kết quả bình thường của nó.



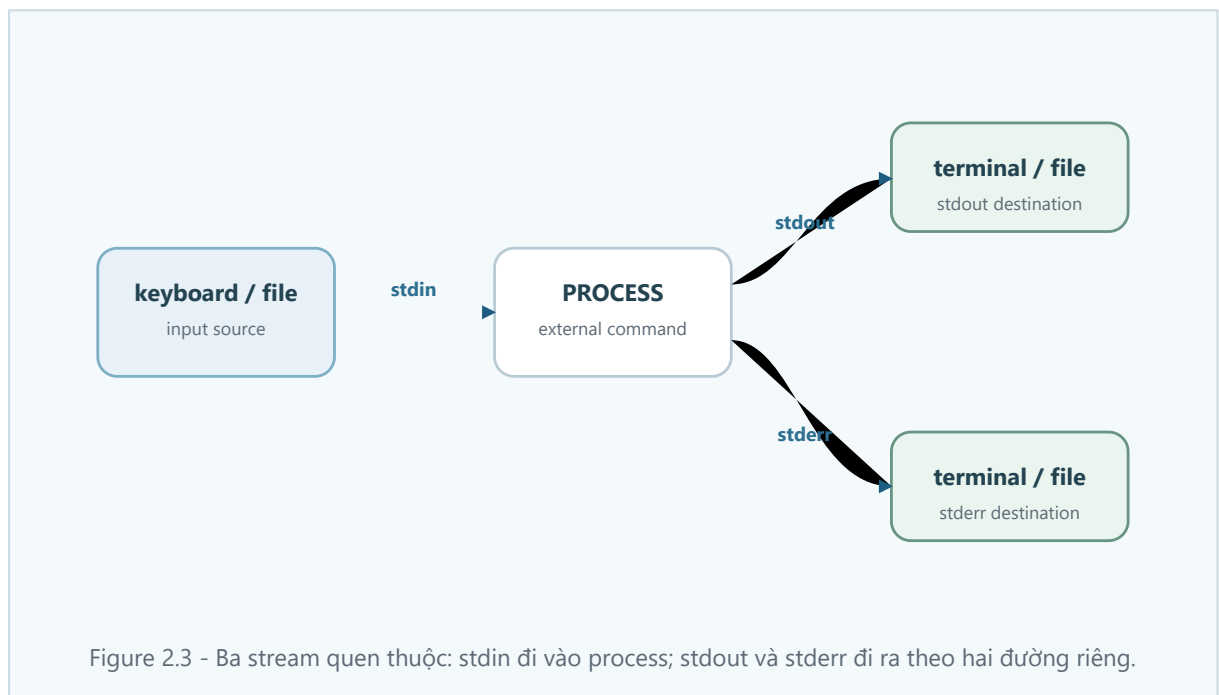
2.5 Standard Error - stderr

Kết quả bình thường không phải là text duy nhất command tạo ra. **standard error (stderr)** là stream riêng thường được dùng cho diagnostic messages: file thiếu, option sai, hoặc permission bị từ chối.

COMMAND

```
cat does-not-exist.txt
```

Message báo lỗi thường hiện cùng terminal với stdout, nhưng không vì thế mà nó là stdout. Hai stream tách riêng để bạn có thể lưu kết quả bình thường mà không trộn diagnostic vào đó.



TRAP

Thấy error message trên màn hình không chứng minh nó là normal output. Giữ stdout và stderr tách riêng cho tới khi bạn chủ động merge chúng.

2.6 File descriptors 0, 1, and 2

Sau khi hiểu ba stream, ta mới cần **file descriptor**. Đây là integer handle mà process dùng để tham chiếu tới một open I/O resource. Nó không phải physical file: một descriptor có thể tham chiếu terminal, regular file, một đầu pipe, hoặc resource đang mở khác.

Descriptor	Stream	Công việc thường gặp
0	stdin	đọc input
1	stdout	ghi normal output
2	stderr	ghi diagnostic output

Shell thiết lập các descriptor/redirection trước khi process chạy. Vì vậy `2>errors.txt` nghĩa là shell đổi destination của descriptor 2, tức stderr; không phải `ls` tự quyết định mọi error phải vào file đó.

2.7 Output redirection: `>`

Quay lại câu hỏi đầu: có thể gửi stdout vào file không? Có. `>` redirect stdout vào file và thay thế nội dung cũ nếu file đã tồn tại.

COMMAND

```
printf 'old line\n' > replace-me.txt
printf 'new line\n' > replace-me.txt
cat replace-me.txt
```

Observation: chỉ còn `new line`. Visual model là `process stdout → file`; technical term là output redirection. Nếu report bị ngắn bất ngờ, dùng ghi thêm, xem `cat report.txt`, rồi kiểm tra bạn có dùng `>` thay vì append không. Trong DevOps, `>` hợp với report mới; không hợp khi bạn phải giữ evidence cũ.

2.8 Append redirection: `>>`

`>>` vẫn redirect stdout, nhưng append vào cuối file thay vì thay thế:

COMMAND

```
printf 'first run\n' > run-history.txt
printf 'second run\n' >> run-history.txt
cat run-history.txt
```

Hai dòng đều còn. Với operational log, append có ích nhưng dễ trộn nhiều lần chạy. Ghi timestamp hoặc run ID khi evidence cần phân biệt từng lần.

TEXT EXAMPLE

```
stdout → file
> replace old contents
>> append after old contents
```

2.9 Input redirection: ``<``

`<` đổi input source của stdin:

COMMAND

```
printf 'alpha\nbeta\ngamma\n' > words.txt
wc -l < words.txt
```

File là input source, `wc` là process, terminal nhận stdout. Nếu file không tồn tại:

COMMAND

```
wc -l < missing-words.txt
```

Shell không mở được input file nên `wc` không nhận stdin như dự định. Diagnostic đi ra stderr. Debug bằng `pwd` và `ls -l`; đừng vội kết luận `wc` đếm sai.

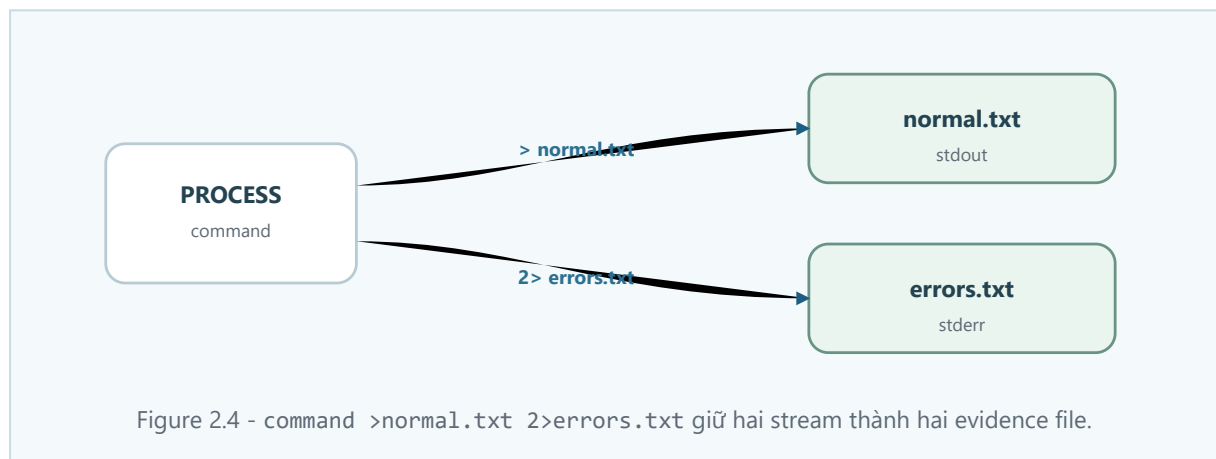
2.10 stderr redirection: `2>`

Để lưu diagnostic riêng, redirect descriptor 2:

COMMAND

```
ls colors.txt does-not-exist.txt > normal.txt 2> errors.txt
cat normal.txt
cat errors.txt
```

`ls` xử lý filename có thật bằng stdout và báo filename thiếu bằng stderr. Sau redirection, `normal.txt` giữ normal output; `errors.txt` giữ diagnostic.



2.11 Gộp stdout và stderr

Khi cần một transcript chung, `2>&1` có nghĩa “đưa stderr (descriptor 2) tới destination hiện tại của stdout (descriptor 1).” Shell đọc redirection từ trái sang phải.

COMMAND

```
ls colors.txt does-not-exist.txt > combined.txt 2>&1
cat combined.txt
```

Ở đây stdout đổi sang `combined.txt` trước; sau đó stderr theo destination hiện tại của stdout. So sánh:

COMMAND

```
ls colors.txt does-not-exist.txt 2>&1 > other.txt
```

Ở lệnh thứ hai, stderr theo stdout cũ (thường là terminal) trước khi stdout chuyển sang `other.txt`. Thứ tự tạo wiring khác nhau. `cmd > file`, `cmd 2> file`, và `cmd > file 2>&1` vì thế phục vụ ba mục tiêu evidence khác nhau.

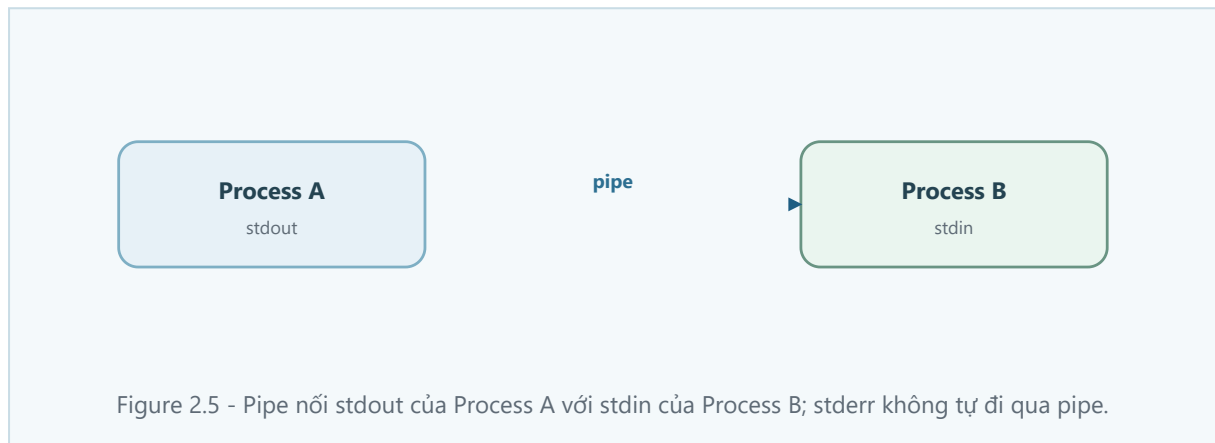
2.12 Pipes: `|`

Nếu không cần giữ file trung gian, một **pipe** nối stdout của producer process vào stdin của consumer process:

COMMAND

```
printf 'alpha\nbeta\nalpha\n' | grep 'alpha'
printf 'alpha\nbeta\nalpha\n' | grep 'alpha' | wc -l
```

`printf` là producer; `grep` lọc text; `wc` là consumer cuối. Pipe không tự động mang stderr. Error từ producer thường vẫn tới terminal, trừ khi bạn chủ động redirect/merge nó.



2.13 Vì sao pipe không phải temporary file?

Pipe chuyển bytes trực tiếp giữa hai process đang chạy. Nó không phải temporary file có tên mà bạn có thể mở sau đó.

COMMAND

```
printf 'one\ntwo\nthree\n' | head -n 2
printf 'one\ntwo\nthree\n' > all-lines.txt
head -n 2 all-lines.txt
```

Cả hai có thể hiện hai dòng. Nhưng route thứ hai cố ý giữ `all-lines.txt` làm evidence; route pipe không tạo file đó. Khi cần raw output cho incident, đừng giả định pipe đã giữ evidence cho bạn.

2.14 Exit status khác error output thế nào?

Diagnostic output và **exit status** là hai signal khác nhau. Sau khi command kết thúc, shell nhận exit status: `0` thường là success, non-zero nghĩa là command báo một điều cần xử lý.

COMMAND

```
grep 'purple' colors.txt
printf 'exit status: %s\n' "$?"
```

Không có match thường làm `grep` trả `1`; điều đó có thể là result hợp lệ, không nhất thiết là command hỏng. Pipeline càng cần thận trọng: consumer cuối có thể tạo output hữu ích dù producer trước đó thất bại.

COMMAND

```
grep 'alpha' missing-input.txt | wc -l
printf 'pipeline exit status: %s\n' "$?"
```

Bạn có thể thấy một số đếm từ `wc` trong khi `grep` đã in diagnostic ra stderr. Vì vậy output đẹp không tự chứng minh mọi stage thành công.

[DEEP-DIVE — BASH PREVIEW]

Trong Bash, `PIPESTATUS` cho biết exit status của từng stage, và `set -o pipefail` thay đổi cách Bash báo pipeline failure. Đây là Bash-specific features; chúng ta sẽ học Bash arrays và shell options đúng cách trong phần Bash for DevOps. Không cần dùng cú pháp đó để pass core lab của chapter này.

2.15 Những lỗi thường gặp và cách debugging

Symptom	Kiểm tra nhỏ nhất	Layer có thể sai
Không thấy text trong <code>report.txt</code>	<code>cat report.txt</code> ; chạy lại không có <code>></code> trong lab	stdout destination
Error vẫn ở terminal	kiểm tra có <code>2></code> chưa	stderr có route riêng
Report cũ biến mất	so sánh <code>></code> với <code>>></code>	replace hay append
<code>grep</code> không in gì	xem <code>\$?</code> ngay sau command	result/status meaning
Pipeline vẫn có count	xem diagnostic và chạy producer riêng	stdout không chứng minh success
File merge thiếu error	đọc <code>2>&1</code> từ trái sang phải	redirection order

Debug theo thứ tự: chạy producer một mình; tách stdout/stderr vào hai file disposable; kiểm tra exit status ngay; thêm từng pipe; chỉ merge khi transcript chung thực sự cần thiết.

2.16 STOP & RECALL

1. stdin có thể nhận input từ đâu ngoài keyboard?
2. stdout và stderr khác nhau ở mục đích nào?
3. Descriptor 0, 1, 2 là gì, và vì sao không phải physical files?
4. `>` khác `>>` thế nào?
5. Vì sao `cmd > all.txt 2>&1` khác `cmd 2>&1 > all.txt`?
6. Pipe nối stream nào với stream nào?
7. Vì sao pipeline có output chưa đủ để kết luận success?

ONE-SENTENCE SELF-CHECK

Vẽ đường đi cho producer `>out.txt 2>err.txt`, rồi nói bạn sẽ xem đâu để biết producer có báo success hay không.

2.17 Bài lab có hướng dẫn

Safety boundary

Chỉ dùng `$HOME/fieldbook-labs/ch02-streams`. Không redirect vào source code, `/var/log`, configuration, hay production log. Không dùng `sudo`.

Mission

1. Tạo `colors.txt` bằng `printf` và đọc lại bằng `cat < colors.txt`.
2. So sánh `>` và `>>` trên `run-history.txt`.
3. Chạy `ls colors.txt does-not-exist.txt > normal.txt 2> errors.txt`; đọc từng file.
4. Chạy `ls colors.txt does-not-exist.txt > combined.txt 2>&1`, rồi chạy `cat combined.txt`; giải thích vì sao normal output và diagnostic output cùng xuất hiện trong file.
5. Chạy command thành công `printf 'alpha\nbeta\nalpha\n' | grep 'alpha' | wc -l`. Sau đó chạy `grep 'alpha' missing-input.txt | wc -l` và ngay lập tức chạy `printf 'pipeline exit status: %s\n' "$?"`. Giải thích vì sao visible count và default Bash pipeline exit status không chứng minh mọi stage thành công.

Core lab không yêu cầu `PIPESTATUS`, Bash arrays, hay `set -o pipefail`.

Evidence	Quan sát thật	Nó chứng minh	Nó chưa chứng minh
<code>normal.txt</code>		stdout destination	toàn bộ command success
<code>errors.txt</code>		stderr destination	stdout contents
<code>combined.txt</code>		deliberate merge	original stream provenance
<code>\$?</code>		status của command/pipeline vừa chạy	mọi stage của Bash pipeline

Rollback chỉ trong lab directory:

COMMAND

```
rm -f colors.txt words.txt numbers.txt message.txt replace-me.txt run-history.txt normal.txt
errors.txt combined.txt other.txt all-lines.txt
```

Kiểm tra `pwd` trước khi cleanup.

2.18 Điểm kiểm tra cuối chapter

Bạn sẵn sàng đi tiếp khi có thể: giải thích input source → process → output destination; dùng `stdin`, `stdout`, `stderr` đúng ngữ cảnh; mô tả FD 0/1/2 là integer handles; chọn `>`, `>>`, `<`, `2>`, `2>&1`, hoặc `|` cho một mục tiêu; và kiểm tra diagnostic cùng exit status thay vì nhìn output một mình.

DevOps connection: Stream reasoning giúp bạn redirect command output vào log đúng mục đích, giữ error output riêng, chain diagnostic commands, và không nhầm pipeline output với bằng chứng mọi stage đã thành công.